COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE21S4 |
| **DATE PERFORMED:** | 09/09/26 | **DATE SUBMITTED:** | 09/15/26 |
| **NAME:** | Hare, Andrei Angelo D. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. 8** **Sorting Algorithms Pt2**

## 1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for implementation of shell, merge and quick sort.

Preparation Task:

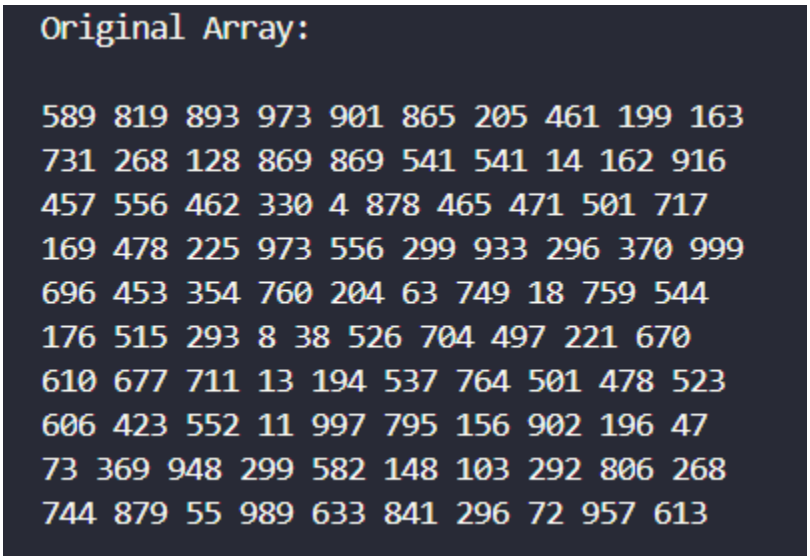
- Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 8-1 in section 6.
- Create a header file for the implementation of the different sorting algorithms. Import this header file into your main.cpp

A.1. Shell Sort
General Algorithm
The general algorithm for shell sort is given below.

A.2. Merge Sort
General Algorithm
The general pseudo-code for the merge sort technique is given below.

A.3. Quick Sort

Code + Console Screenshot

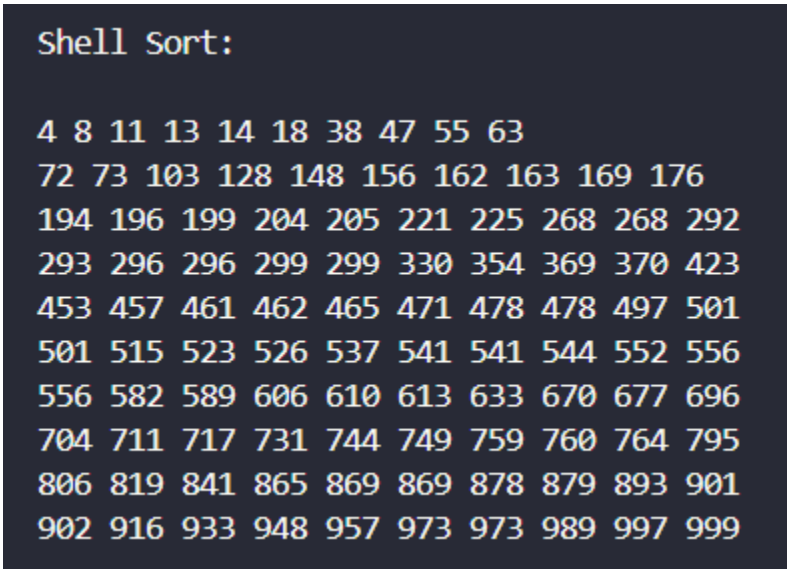



Observations

The program successfully generated 100 random integer values. The values were initially unsorted and were used as the original array for testing the three sorting algorithms.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot



| |
|---|
| Observations<br>It successfully arranged the 100 values in ascending order by comparing elements using different gaps. |

Table 8-2. Shell Sort Technique

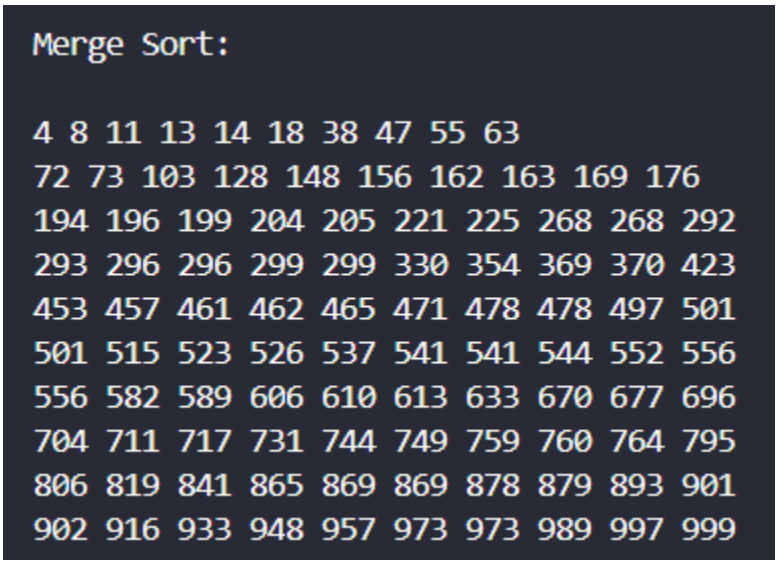
| |
|---|
| Code + Console Screenshot<br>  |
| Observations<br>It successfully sorted the 100 values in ascending order by dividing the array into smaller parts and merging them in the correct order. |

Table 8-3. Merge Sort Algorithm

| |
|---|
| Code + Console Screenshot |

```
Quick Sort:

4 8 11 13 14 18 38 47 55 63
72 73 103 128 148 156 162 163 169 176
194 196 199 204 205 221 225 268 268 292
293 296 296 299 299 330 354 369 370 423
453 457 461 462 465 471 478 478 497 501
501 515 523 526 537 541 541 544 552 556
556 582 589 606 610 613 633 670 677 696
704 711 717 731 744 749 759 760 764 795
806 819 841 865 869 869 878 879 893 901
902 916 933 948 957 973 973 989 997 999
```

Observations
It successfully sorted the 100 values in ascending order by using a pivot to partition the array into smaller parts.

Table 8-4. Quick Sort Algorithm

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms
Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Answer:
Yes, we can use other sorting algorithms for the left and right sublists from Quick Sort. For example, if we have {8, 3, 6, 2, 9, 5, 1} and use 5 as the pivot then the array can be divided into {3, 2, 1} | 5 | {8, 6, 9}. We can use Insertion Sort to sort the left and right sublists. The left side becomes {1, 2, 3} and the right side becomes {6, 8, 9}. After putting them together with the pivot, the final sorted array is {1, 2, 3, 5, 6, 8, 9}. So i believe Sir yes, other sorting algorithms can also be used for the sublists in Quick Sort.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Answer:

For the given array, I believe Quick Sort can provide fast performance because it uses a pivot to divide the array into smaller parts and then sorts those parts. But its performance can depend on how the pivot is selected. Merge and Quick Sort can have O(n log n) time complexity because both use a divide and conquer approach. Merge Sort divides the array into smaller parts and then merges them back together. Quick Sort divides the array based on a pivot and sorts the smaller sublists. So when the divisions are balanced then both algorithms can process the elements in O(n log n) time. Quick Sort can become $O(n^2)$ in the worst case when the pivot creates very unbalanced partitions. Merge Sort on the other hand maintains the O(n log n) time complexity even in its worst case.

COMPUTER ENGINEERING

QUEZON CITY

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Quillbot, Google Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar correction and summarization**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **AI tools were used strictly for grammar and spelling checks**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.